

MLOps PROJEKAT · 2026

AMR Fleet Monitoring

MLOps arhitektura za praćenje flote autonomnih mobilnih robota

Predikcija otkaza iz telemetrije · Serving (FastAPI + Docker) · Drift monitoring (EvidentlyAI)

Sara Babić

Jul 2026.

Pregled izlaganja

► 1 · Teorijski deo (~5 min)

Zašto monitoring modela · Data i Concept Drift · Nesupervizovana detekcija

AI observability (tri sloja) · Automatizacija re-treninga

► 2 · Praktični deo (~8 min)

Kompletan MLOps tok: podaci → model → REST serving → drift monitoring

Docker kontejnerizacija i mašinski čitljiva odluka za automatski retrening

► **Ključni rezultati:** PR-AUC 0.82 · model 38× manji bez gubitka tačnosti · monitor detektuje ubačeni kvar, bez lažnih alarma na zdravom saobraćaju

Zašto je monitoring modela u produkciji neophodan

- ▶ Trening pretpostavlja:
 - raspodela podataka i veza ulaz → ishod ostaju **stabilni** i posle puštanja u produkciju
- ▶ U realnom sistemu okruženje se menja:
 - raspored magacina · tip tereta · gustina saobraćaja robota · habanje senzora · sezonski obrasci
- ▶ Posledica: performanse **tiho opadaju** → model i dalje vraća predikcije, samo netačne
- ▶ Bez monitoringa se problem primeti tek kada šteta već nastane

Rešenje: sistemsko praćenje modela u produkciji → model monitoring, odnosno , AI observability.

Data drift naspram concept driftu

DATA (VIRTUELNI) DRIFT

Menja se raspodela ulaznih podataka $P(X)$

npr. drugačiji obrazac telemetrije zbog habanja senzora; veza ulaz $\rightarrow$ ishod ne mora da se promeni

CONCEPT (REALNI) DRIFT

Menja se sama veza ulaz $\rightarrow$ ishod $P(Y|X)$

npr. ista telemetrija više ne znači isti rizik otkaza, model uči zastarelu relaciju

- **Formalna definicija (Hinder, Vaquet i Hammer, 2024):** drift postoji $\Leftrightarrow$ podaci X i vreme posmatranja T **nisu statistički nezavisni**
- **Vremenska dinamika:** nagli · postepeni · inkrementalni · ponavljajući (sezonski)

Detekcija drifta bez ground-truth oznaka

- ▶ **Problem:** stvarni ishod zadatka (Y) u produkciji stiže sa zakašnjenjem ili nikad → nadgledana detekcija je nerealna
- ▶ **Rešenje:** nesupervizovana detekcija → prate se raspodela ulaza i raspodela predikcija

Prediction drift = najraniji vidljivi simptom concept drifta (proxy signal)

- ▶ **Opšta šema detektora (Lu i sar., 2018):** prozori podataka → deskriptor raspodele → mera nesličnosti → statistička odluka
- ▶ **Familije metoda:**
 - two-sample testovi nad dva prozora: Kolmogorov-Smirnov, PSI, MMD ← *koristi ih Evidently*
 - meta-statističke (ADWIN) i block-based (DAWIDD): preciznije, manje lažnih alarma

AI observability: tri sloja praćenja

SLOJ 1

Kvalitet i drift ulaznih podataka

raspodela feature-a naspram trening (referentne) raspodele

SLOJ 2

Performanse modela

kvalitet predikcija · latencija inferencije · stabilnost izlaza

SLOJ 3

Operativni / poslovni uticaj

uspešnost izvršenja zadataka flote · kašnjenja · intervencije operatera

Preskočen sloj = slepa tačka: API radi i kontejner je „zdrav“, dok model tiho daje sve lošije predikcije. Log svih zahteva je istorijska osnova koja povezuje slojeve.

Automatizovan re-trening i specifičnost AMR domena

SCHEDULED (fiksni interval)

- + jednostavan za implementaciju
- rasipa resurse ili kasni sa reakcijom

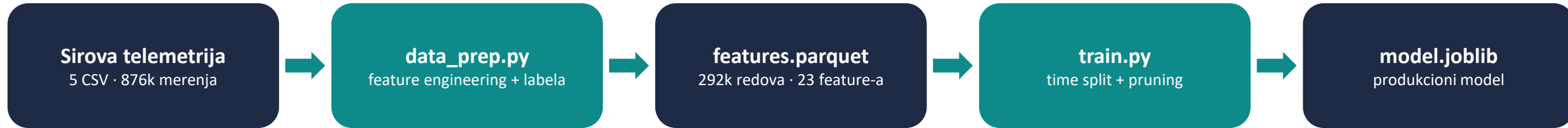
EVENT-DRIVEN (trigger na drift)

- + reaguje odmah na stvarnu promenu
- rizik lažnih alarma → cooldown mehanizam (AutoDrift)

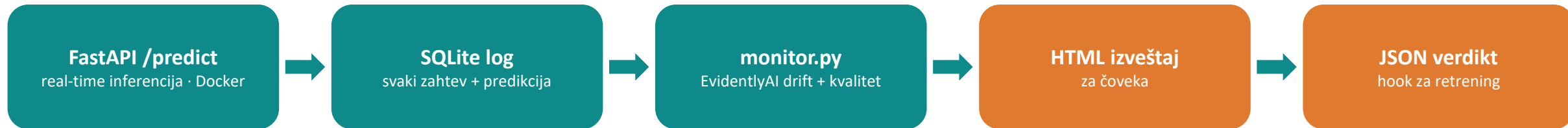
- **Multi-criteria pristup:** retrening samo kada očekivano poboljšanje opravdava računarski trošak
- **AMR flote su specifične:** bezbednosno kritične · heterogeni senzori u realnom vremenu · okruženje se fizički menja

Arhitektura sistema: kompletan MLOps tok

OFFLINE: priprema i trening



ONLINE: serving i monitoring



- **Dve MLOps celine implementirane end-to-end:** serving & deployment + monitoring & observability
- **replay.py** simulira produkcionu saobraćaj (sa opcijom ubacivanja sintetičkog kvara)

Podaci: strukturni proxy za AMR telemetriju

- ▶ Realna AMR telemetrija je uglavnom u privatnom vlasništvu kompanija; javni AMR skupovi nemaju outcome label
- ▶ Izbor: **Microsoft Azure Predictive Maintenance** (Kaggle) set podataka gde je ista topologija monitoringa:

$100 \text{ jedinica} \times 8.761 \text{ h} \times 4 \text{ senzora} = 876.100 \text{ merenja tokom cele godine} + 761 \text{ stvarni otkaz (label)}$

Signal u skupu	AMR analogija	Uloga
volt · rotate · pressure · vibration	struja motora · odometrija · opterećenje aktuatora · vibracije šasije	ulazni feature-i (drift)
error1-error5	fault kodovi robota	kvalitet podataka + signali
age, model	starost i varijanta robota	statički feature-i
otkaz komponente (comp1-4)	kvar podsistema → neizvršen zadatak	ciljna labela

Odbačena alternativa: „Logistics & Supply Chain“ skup → sintetički, slaba temporalna struktura, gotove risk kolone → target leakage. Azure skup tera da labelu izgradimo sami.

Priprema podataka i definicija cilja

► **23 feature-a po jedinici i vremenskom trenutku:**

- 16 rolling statistika: mean i std svakog senzora u prozorima od 3 h i 24 h (kratkoročno stanje + varijabilnost)
- 5 brojača grešaka u poslednja 24 h · 2 metapodatka (model, starost)

► Prozori se računaju po jedinici, tako nema curenja podataka između mašina; uzorkovanje na svaka 3 h → 292.000 redova

► **Labela: failure_within_24h** → da li jedinica otkazuje u narednih 24 h („jedinica neće izvršiti zadatak“)

► Samo **1,96 % pozitivnih** → jako nebalansirane klase → oblikuje ceo trening

Trening i kompresija modela (pruning)

- ▶ **Random Forest** · vremenski split (poslednjih 20 % vremena = test) · random split bi procureo budućnost u trening
- ▶ **class_weight=balanced** · metrika PR-AUC · accuracy je besmislen kod 2 % pozitivnih
- ▶ **Cost-complexity pruning (ccp_alpha)**: najjače orezivanje koje čuva ≥ 98 % baseline PR-AUC

Metrika	Baseline	Pruned (produkcija)	Promena
PR-AUC	0.814	0.818	kvalitet očuvan
Broj čvorova	368.490	9.214	40× manje
Veličina na disku	29,5 MB	0,78 MB	38× manji
Latencija po zahtevu	15,5 ms	15,6 ms	nepromenjena

Zaključak: dobitak pruninga je deployment footprint (kontejner, RAM, cold-start), a ne brzina jer latencijom dominira Python/sklearn overhead, ne obilazak stabala.

Serving: FastAPI servis i Docker slika

Endpoint	Namena
GET /health	liveness + da li je model učitao + prag odluke
POST /predict	verovatnoća otkaza za jedan vektor telemetrije (+ upis u log)
GET /stats	broj logovanih predikcija (sanity check monitoringa)

- Pydantic validira svaki zahtev; prag odluke konfigurabilan kroz **DECISION_THRESHOLD** env promenljivu
- Artefakt = ceo sklearn Pipeline (one-hot enkoder + šuma) → **nema train/serve skew-a**, serving ne duplira nikakvu pripremu podataka
- Svaki zahtev se upisuje u SQLite (vreme, ulazi, verovatnoća, odluka) → istorijska osnova za monitoring
- **Docker:** lean slika (python 3.10-slim + samo serving zavisnosti; sklearn pinovan na trening verziju); volume mount da log ostane na hostu

Monitoring: EvidentlyAI drift detekcija

REFERENCA

trening period: kako izgleda „normalno“, uključujući raspodelu predikcija modela

TRENUTNI PROZOR

najnoviji produkcionni zahtevi, pročitani iz SQLite loga

- ▶ **Evidently računa:** data drift po koloni · prediction drift · kvalitet ulaznih podataka
- ▶ **Pravilo alarma - `decide()`:** dataset-level drift $\vee \geq 30\%$ kolona driftovalo $\vee$ prediction drift
 - prediction drift je najvažniji okidač: promenjen izlaz znači promenjeno ponašanje modela, bez obzira koji se ulaz pomerio
- ▶ **Dva artefakta:** interaktivni HTML izveštaj (za čoveka) + JSON verdikt (za mašinu - tačka gde bi se zakačio automatski retrening)

Rezultati

Scenario	Driftovane kolone	Prediction drift	Verdikt
A: zdrav saobraćaj (pravi kasniji podaci)	0 (0 %)	ne (skor $\approx 0,04$)	nema alarma
B: ubačeni kvar flote (povišene vibracije i pritisak)	7 (29,2 %)	da (skor $\approx 1,18$)	DRIFT DETEKTOVAN

► **Kompletan monitoring u dva scenarija:**

- nema lažnog alarma na stvarnim kasnijim podacima (telemetrija je stacionarna → ispravan negativan rezultat)
- alarm se pali tačno kada je kvar ubačen → pozitivna kontrola da detektor radi

► U JSON verdiktu:

"drift_alert": true, "reasons": ["prediction (output) drift"]

► Isti scenario ponovljen i unutar Docker kontejnera (uz volume mount koji daje log fajl host mašini)

Zaključak

Koncept iz literature	Implementacija u projektu
Sloj 1 observability-ja (data/prediction drift)	EvidentlyAI monitor nad SQLite logom
Istorijski sloj podataka	SQLite log svakog zahteva (FastAPI)
Nesupervizovana detekcija (nema ground truth-a)	prediction drift kao proxy signal
Event-driven retraining	mašinski čitljiv JSON = spreman trigger za retreniranje

- **MLOps ciklus:** podaci → model (PR-AUC 0.82) → serving (FastAPI + Docker) → monitoring → spremna tačka za retrening
- **Ključne brojke:** 38× manji model bez gubitka tačnosti · 0 lažnih alarma · ubačeni kvar detektovan
- **Dalji rad:** automatski retrening pipeline sa cooldown mehanizmom · metrike Sloja 3 (uspešnost zadatka flote)

Hvala na pažnji, pitanja?